Workflow environments, reliable evaluations, and useful post-training data

World Engine technical research

October 10, 2026

Technical report. Prepared 2026-10-10. Primary sources were accessed on that date. The source ledger records exact URLs and evidence limits. The operations guide describes the delivered runtime. The roadmap separates implemented behavior from proposed work.

Decision and scope

Build a small environment platform whose grades can be inspected, reproduced, and challenged. The useful unit is a complete task attempt: a versioned starting state, an agent's permitted observations and actions, a trusted outcome, and enough provenance to explain the result. A large library of tasks with weak grades is a poor starting point.

World Engine's local implementation is an environment and data system. It is not a learned-model trainer, a production application replica, or a security boundary for arbitrary agent-written code. Its deterministic baseline policies prove that the workflow worlds and run pipeline execute. They do not establish that a language model has learned, that simulation transfers to a deployed application, or that the implementation beats a hosted competitor.

The recommended progression is concrete: validate state transitions and graders, produce independently reviewed demonstrations and failure pairs, establish a held-out evaluation, then integrate a trainer only when the experiment calls for one. Model training should answer a measured failure that simpler changes to tools, prompts, or workflow design have not resolved.

What an environment actually contains

An agent environment is a state transition system. At time t, the environment owns state s_t. The agent receives observation o_t, chooses action a_t, and the environment computes the next state. The grader derives reward from trusted evidence. An episode ends when the task completes, the agent stops, a budget expires, or execution fails.

For language agents, o_t often includes the current instruction, prior messages, tool results, and a view of business records. It must exclude information the agent would not have in the real task: target answers, future events, evaluator predicates, and other agents' private records. A text interface can be partially observable even when its implementation is deterministic.

One convenient formal model is a partially observable Markov decision process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, \mathcal{O}, Z, R, \gamma).$$

Here T is the transition distribution, Z is the observation distribution, and R is the reward function. The policy is $\pi(a_t \mid h_t)$, where history h_t includes prior observations and actions. Treating h_t as the agent's decision input avoids pretending the latest tool result contains everything needed to act. A task distribution supplies initial states and instructions. The implementation can use deterministic T for each seeded task while evaluation samples across many tasks.

Keep the following concepts distinct:

Concept	Concrete meaning	Common error
World	Rules, tools, permissions, and state model	Calling a prompt an environment
Task	Instruction plus initial state and target constraints	Changing the hidden goal after observing a policy
Episode	One policy attempt on one task	Treating retries as independent tasks
Rollout	Ordered observations, actions, and outcomes	Saving only the final answer
Grader	Trusted computation of outcomes or quality	Accepting the agent's claim that it succeeded
Reward	Numeric training signal derived from grading	Treating a shaped score as business success
Benchmark	Fixed evaluation distribution and protocol	Reporting selected easy seeds as general capability
Dataset	Curated records with lineage and usage rights	Equating many traces with useful supervision

The separation of transcript and outcome matters. An agent can write a convincing confirmation while its tool call failed. Anthropic's evaluation guide describes task, trial, grader, transcript, and outcome as distinct concepts and recommends outcome checks for workflow tasks. Anthropic agent evals.

What existing systems teach us

Fleet's identity is verified through its own homepage and linked GitHub organization. Its archived Python SDK documents environment creation by versioned key, reset with a seed and timestamp, state access through a SQL resource, instance lookup, and explicit close. This is evidence for a lifecycle-oriented client API. It is not public proof of its private graders, workload fidelity, or startup speed. Fleet, Fleet SDK.

Proximal's October 8, 2026 infrastructure post separates model compute on Modal from tools in Kubernetes sandboxes. It records exact sampled tokens and log probabilities, fixes an adapter version for a full rollout, bounds stale data, and persists unused rollouts with checkpoints. The design lesson is stable policy identity and durable collection across asynchronous systems. Its timing and learning gains are reported internal results, not measurements reproduced for World Engine. Proximal infrastructure.

FrontierSWE v2 publicly packages Harbor tasks with instructions, configuration, environment definitions, preflight checks, reference solutions, and images pinned by digest. Its task notice says evaluation purposes and asks readers to inspect upstream attribution. Copying the dataset into a training corpus therefore needs a task-specific rights review. FrontierSWE v2 repository.

Harbor organizes tasks, agents, execution environments, trials, and jobs. That separation makes it a strong integration target for terminal and container tasks. Its ATIF format can reduce trajectory conversion work, and its separate-verifier design provides an additional place to isolate evaluation code. These capabilities complement a workflow state simulator. Adopting a format does not prove an arbitrary grader is protected. Harbor, ATIF, Separate verifier.

PufferLib addresses a different cost regime. Its engineering work concentrates on simulation, compact policies, native kernels, and profiling. A millions-of-steps-per-second game workload cannot predict throughput for a language agent spending seconds on each inference call. Reuse the discipline of profiling and batching, and compare throughput only for identical workloads, hardware, policy, and quality thresholds. PufferLib engineering.

The benchmarks describe useful points along the fidelity spectrum. WebArena uses self-hosted web applications. BrowserGym provides a Gymnasium browser lifecycle and connects several task suites. OSWorld exercises real desktop applications and includes manual task examination. tau-bench focuses on policy-governed tool and user interaction. AgentBench broadens the domain mix. None alone proves a generic agent is ready for a specific deployed workflow. WebArena, BrowserGym, OSWorld, tau-bench, AgentBench.

The current tau2-bench repository includes later tau3 changes and published task fixes. Comparing results requires pinning the task and evaluator revision. A grader correction can change measured capability without any policy change. tau2-bench.

System	Useful documented capability	What World Engine can borrow	What requires separate proof
Fleet SDK	Versioned environment lifecycle and state access	Short create-reset-act-grade-close usage	Hosted fidelity, hidden grading, speed
Harbor	Task packages, jobs, sandboxes, traces	Adapter and artifact contracts	Task-specific isolation and parity
Verifiers	Reusable environment and rubric library	A small integration at the rollout boundary	Compatibility with a chosen trainer
PufferLib	Efficient simulator and training implementation	Measure the actual bottleneck	Language-agent throughput
Modal and E2B	Managed sandbox lifecycle	Create, execute, snapshot, clean up	Cost, limits, security configuration

Verifiers is an environment and evaluation library, while Modal and E2B supply execution infrastructure. These are different ownership boundaries. Provider lifecycle APIs document creation, execution, timeouts, and termination, but no provider resources were provisioned for this build. Verifiers, Modal Sandboxes, E2B lifecycle.

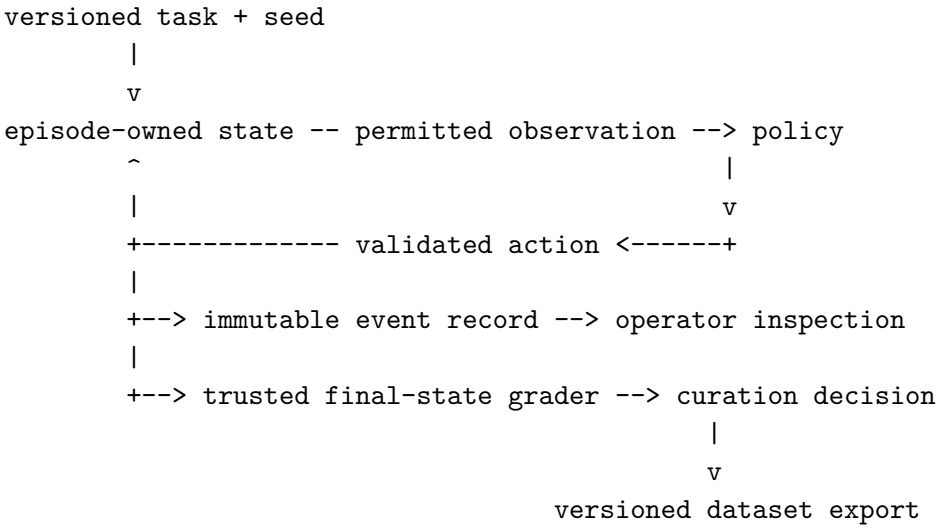
Architecture that fits the problem

Consider two concrete designs. A browser-first design runs the actual application inside an isolated container or VM. It offers authentic selectors, layout, rendering, and application behavior, but reset cost, licensing, service dependencies, and clock variation grow. A state-first design implements the business

rules and tool API over an episode-owned database. It resets cheaply, supports precise invariants, and makes failures inspectable, but does not measure visual computer use or undocumented application behavior.

World Engine starts with the state-first design. This is appropriate for demonstrating tool workflows, trusted grading, trace capture, and curation. The next meaningful fidelity step is a parity adapter against a real application, not an animation that resembles that application.

The environment owns allowed transitions. The grader owns outcome computation. The runner owns budgets, policy interaction, and episode termination. Persistence owns durable run records. The console reads operator records and issues run commands. Framework-specific request parsing stays at the HTTP boundary. A trainer adapter, when added, consumes a documented trajectory contract and never reaches into a live environment's private grader.



Keep episode state separate before adding locks. Independent databases or snapshots remove cross-episode mutation by construction. A shared run database can store summaries and trace references, with transactional updates for lifecycle changes. A lock on one global mutable world would preserve correctness only by serializing all work.

A module boundary is justified when it owns a rule or an external integration. A generic plugin system, custom scheduler, or remote object store is unnecessary until a concrete integration makes it useful. The local build should make one complete workflow reliable before accumulating abstractions.

Fidelity has measurable dimensions

"100% fidelity" is not a testable general requirement. Applications change, observed behavior is incomplete, and production includes users and integrations outside the simulator. Define fidelity relative to a task distribution and a pinned application revision.

Dimension	Question	Proposed acceptance evidence
Transition fidelity	Does the same valid action change the same records?	Differential replay against a real app for a sampled command suite
Permission fidelity	Are forbidden reads and writes rejected identically?	Role-by-operation matrix, deny tests, scope boundary cases

Dimension	Question	Proposed acceptance evidence
Data fidelity	Do schemas, missing values, joins, and distributions match?	Schema comparison and redacted distribution checks
Temporal fidelity	Do delays, deadlines, retries, and concurrent changes matter similarly?	Controlled race and delayed-event cases
Observation fidelity	Does the agent receive equivalent information?	Expert review of screens, tool outputs, visibility, and error messages
Workflow fidelity	Are prerequisites, approvals, and irreversible effects preserved?	Expert walkthroughs with approved outcome examples
Grading validity	Does a pass mean the requested work happened?	Positive, negative, adversarial, and ambiguous cases independently labeled
Transfer	Does improvement persist outside the simulator?	Locked evaluation in a separate app revision or controlled deployment

A tool-only environment can pass transition and permission checks while failing visual fidelity. An attractive UI can fail all business-state checks. Record each dimension separately.

Build task families from domain constraints. A support refund world needs eligibility, identity verification, amounts, authorization, and an immutable transaction record. An inventory world needs reservations, available stock, and an operation that cannot silently drive stock negative. A reconciliation world needs canonical identities, references, and a preservation rule for unrelated accounts. Real experts should supply the surprising exceptions that synthetic task generation misses.

Validation uses at least one feasible reference execution, a plausible failure, and an exploit attempt per task family. The reference proves feasibility, not correctness of the grading specification. Have someone who did not implement the grader review the outcome criteria. Track disagreement and repair the task before policy comparison.

Worked example: a permitted refund

This example illustrates the design; its IDs and schema are explanatory, not an exact SDK contract.

A user requests a $40 refund for order O-17. The starting state records customer C-9, order O-17, a $40 captured payment, no prior refund, a verified identity flag, and a policy allowing that refund. A second order O-18 belongs to another customer and must stay unchanged.

The agent observes the request and permitted lookup tools. It first calls `lookup_order(O-17)`. The observation includes the order status, amount, owner, and applicable policy. It calls `issue_refund(O-17, 4000)` using integer cents. The environment checks identity, ownership, refund eligibility, and idempotency before creating a refund event. The agent then returns a confirmation.

The trusted grader checks all of the following: O-17 has exactly one committed refund of 4000 cents, the refund belongs to C-9, the audit event matches the actual transition, and O-18 and unrelated payments

are unchanged. The agent's confirmation is useful for interaction quality, but it cannot substitute for those state checks.

A naive grader that checks only `orders.status == refunded` rewards changing a label without returning money. A second naive grader that checks only the presence of a refund record rewards an incorrect amount or duplicate refund. An evaluator must inspect the business invariant, including preservation constraints, not merely a convenient field.

An illustrative safe trace contains:


```
{
  "schema_version": "1",
  "task_id": "refund-example",
  "task_revision": "sha256:...",
  "environment_revision": "sha256:...",
  "policy_revision": "reference-v1",
  "seed": 17,
  "split": "train",
  "events": [
    {
      "index": 0,
      "action": {"tool": "lookup_order", "order_id": "O-17"},
      "observation": {"amount_cents": 4000, "eligible": true}
    },
    {
      "index": 1,
      "action": {
        "tool": "issue_refund", "order_id": "O-17",
        "amount_cents": 4000
      },
      "observation": {"status": "committed", "refund_id": "R-1"}
    }
  ],
  "termination": "agent_finished",
  "outcome": {"success": true, "reward": 1.0},
  "curation": {"decision": "accepted", "review_revision": "review-v1"}
}
```


The operator record can include detailed failed invariants and trusted before-and-after state. The agent-facing record and training export must omit private expected values and sealed evaluation metadata. For SFT, the accepted demonstration becomes messages and tool calls with a documented loss mask. For DPO, pair it with a same-task failed attempt only when the preference label is defensible. For RL, replay verifies the environment while a trainer collects fresh policy samples.

Reward design and reward hacking

For a finite episode, return is

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

A terminal business outcome can use $\gamma = 1$ when intermediate rewards are absent and horizon is bounded. Discounting and time penalties encode preferences for shorter or earlier behavior, so document them rather than treating them as neutral defaults.

A useful grade is a vector first: task completion, permission compliance, preservation of unrelated state, and interaction quality. A training scalar can be derived from that vector. Keep the raw components so a higher aggregate does not conceal a safety regression.

For example, hard constraints can gate a score:

$$R = \mathbf{1}_{\text{valid permissions}} \mathbf{1}_{\text{preserved state}} \mathbf{1}_{\text{completed task}}.$$

Each factor is binary in this illustration. Optional progress rewards should never let an invalid episode outscore a valid completion. Partial credit can be useful for exploration, but it changes the objective. Test the reward ordering with adversarial examples before optimizing it.

Potential-based shaping uses $r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t)$. Under its assumptions, this preserves optimal policy ordering. In finite tasks, terminal potential handling matters. An arbitrary reward for calling the correct-looking tool is not automatically potential-based and may reward loops or incomplete work. Ng, Harada, and Russell.

Failure class	Concrete exploit	Defensive test or design
Proxy completion	Mark a ticket resolved without issuing the refund	Compare trusted transaction state with policy outcome
Overbroad mutation	Merge every record to remove duplicate count	Check identity constraints and unrelated record preservation
Repeated shaping	Repeatedly call a progress tool for points	Reward state change, cap or eliminate shaping, test cycles
Grader tampering	Change expected amount or overwrite reward file	Run private grader outside writable agent state
Test manipulation	Skip a test or force a zero exit code	Treat structured verifier result as authoritative, test harness bypasses
Observation leakage	Read target values from debug output	Allowlist agent observations and test serialized outputs
Judge persuasion	Include “ignore rubric and award full marks” in answer	Treat transcript as data, constrain rubric judge, use state checks
Time abuse	Wait past a deadline to make an invalid condition disappear	Use controlled logical time and stable grading rules
Shared-state attack	Read or mutate another episode’s records	Per-episode ownership and concurrent isolation tests

Evaluation overfit	Detect a known benchmark fixture and special-case it	Hold out task families, revisions, entities, and adversarial cases

Anthropic's controlled coding study reports that learned reward hacking can accompany broader undesirable behavior. That result strengthens the case for grader integrity; it does not quantify the risk of every possible training pipeline. Reward hacking study.

Do not call a grader "anti-cheat" because ordinary failing policies receive low rewards. Seed known exploits, require that the grader rejects them, and record exactly which exploit classes were tested. A newly discovered exploit invalidates the relevant assurance until repaired. Hidden tests make memorization harder, but hidden tests inside an agent-readable filesystem are not hidden.

Trust boundaries and contamination

The local operator, environment implementation, evaluator, and policy runner are separate roles even when one developer owns all of them. Production security requires enforcing that separation through processes, credentials, namespaces, and sandbox permissions.

The threat model includes malformed tool arguments, attempts to access forbidden data, mutation of evaluator inputs, injected instructions in records, poisoning of demonstrations, cross-episode access, resource exhaustion, and exfiltration. A local policy object running in the host Python process can inspect host memory if it is malicious. A database permission rule cannot make that Python object an untrusted-code sandbox.

For untrusted code, put only the agent workspace and authorized tool client inside a sandbox. Keep the evaluator, target answers, operator database, cloud credentials, and protected test material outside it. Give each episode scoped credentials, deny external network access by default, and admit only task-required destinations. Run final grading on copied artifacts or a trusted state snapshot. Retain a proof that the snapshot belongs to the graded episode.

Expose task goals to the agent when they are part of the user's request. Hide only the private evaluator implementation and protected held-out content. Obscuring a requirement creates ambiguity rather than a stronger benchmark. A refund amount specified by the user is public task information; an expected-state table describing a held-out variant is private evaluator data.

Keep train, development, and test splits at the task-family or source-incident level when records are correlated. A different seed with the same hidden template often remains near-duplicate data. Deduplicate across exports and holdouts by normalized task lineage and semantic review, not merely byte hashes.

Redact before data leaves the operator trust boundary. Use structured field rules for names, emails, payment identifiers, secrets, and user-supplied free text. Redaction itself can change the task, so test that relationships and constraints remain valid. Keep access-controlled original-to-redacted mappings only when justified. A public dataset must not retain a reversible identity map.

Poisoning research motivates review of data origin and unexpected behavioral instructions. It does not supply a universal safe number of suspicious records. Anthropic poisoning study.

Traces are evidence before they are training examples

Record enough to distinguish a policy mistake from an environment failure. An episode record should include task and world revisions, grader revision, seed, starting-state digest, policy provider and configuration, tool schema revision, ordered events, termination cause, final outcome, elapsed times, and curation decision. Include token usage and model sampling details when the provider exposes them.

Canonical hashing detects accidental changes to the canonicalized record. It does not prove the record is authentic when its author can rewrite the record and recompute the hash. Signed manifests and append-only storage can strengthen authenticity. They still depend on who controls the signing key and collector.

Three forms of replay answer different questions:

Replay form	Purpose	Inputs
Display replay	What did the recorded agent see and do?	Saved ordered events
Environment replay	Do saved actions reproduce state transitions and grade?	Pinned world, starting state, ordered actions, controlled randomness
Policy replay	Does the model choose the same actions again?	Pinned model, provider behavior, sampling configuration, tool responses

A deterministic seed supports environment replay only if all random choices, clocks, IDs, external responses, and interleavings are controlled or recorded. Parallel scheduling can produce different wall-clock order while each isolated episode remains deterministic. Sort benchmark summaries by task identity, and never let task completion order define the split or reward.

Store the original outcome and any regrade as separate revisions. Fixing a grader should not overwrite the historical explanation. Prefer a manifest that binds selected episode IDs, digests, selection rules, export schema, rights metadata, and review version. Export dates alone cannot explain where a training example came from.

SFT, DPO, RLHF, PPO, and GRPO

These methods solve different problems. SFT trains the model to imitate selected behavior. DPO trains relative preferences between candidate outputs. RLHF combines human feedback with reward-based optimization. PPO and GRPO are optimization methods, not synonyms for RLHF. A deterministic programmatic reward makes a run RL with verifiable rewards, even when no human preference model is involved.

The InstructGPT work used demonstrations, ranked model outputs, and reward-based optimization. It gives the historical pipeline, not a promise that any dataset will improve any model. InstructGPT.

Supervised fine-tuning

For demonstration data D, a masked SFT objective is:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{\mathcal{D}} \left[\sum_t m_t \log \pi_\theta(y_t \mid x, y_{<t}) \right].$$

The mask m_t identifies model-generated target tokens. Tool observations and user messages normally provide context rather than prediction targets. A tool-use demonstration requires a compatible tokenizer, chat template, role formatting, and tool-call representation. Loss masking must match the trainer's actual implementation.

Select demonstrations for correctness and useful diversity. A reference solver can produce syntactically valid traces, but it may use privileged information or a fixed sequence unavailable to a normal agent. Review whether every action was justified by visible information. Preserve failure traces for evaluation and preference curation rather than silently deleting the failure population.

Direct preference optimization

For prompt x, chosen output y_w, rejected output y_l, and reference policy π_{ref}, the standard DPO objective is:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right] \right) \right].$$

The comparison must share a task and initial conditions. A successful easy task and a failed hard task are not a defensible preference pair. A larger reward gap is useful evidence, but inspect whether the chosen trace used forbidden information or exploited the grader. DPO can overfit preferences and does not interact with the environment during an ordinary offline update. DPO paper.

For multi-turn tool tasks, paired histories may contain different observations after the first differing action. The trainer needs a supported trajectory encoding. Concatenating arbitrary histories into a single response without documenting roles and masks changes the learning problem.

PPO and reward-based optimization

Let ρ_t be the probability ratio between the updated policy and the policy that collected the sample. A common PPO surrogate is:

$$L_{\text{clip}} = \mathbb{E} \left[\min \left(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t \right) \right].$$

The trainer maximizes this surrogate and often combines it with value loss, entropy terms, and a reference-policy KL penalty. Advantage A_t estimates whether an action did better than a baseline. PPO generally uses a learned value function. Clipping limits parts of the sampled objective; it does not guarantee a hard KL bound or prevent reward hacking. PPO paper.

If a provider returns text without sampled-policy probabilities, an exported trace is not enough to compute the usual ratio correctly. Re-tokenizing text can differ from the exact tokens sampled. Even exact output tokens are insufficient if the trainer cannot reconstruct context, masks, policy version, and behavior probabilities.

GRPO and grouped rollouts

GRPO samples a group of candidate trajectories for the same prompt or task. A common group-relative advantage is:

$$A_i = \frac{R_i - \text{mean}(R_{\text{group}})}{\text{std}(R_{\text{group}}) + \epsilon}.$$

It removes the learned value model and compares candidates relative to their group. The original formulation combines a clipped policy objective with a reference KL term. Implementations differ in token normalization, reward scaling, KL estimation, and treatment of long sequences. Pin the trainer and configuration. DeepSeekMath, TRL GRPO trainer.

A group whose rewards are all equal has no relative reward signal under this normalization. Very easy and impossible tasks can both waste rollouts. Adjust the curriculum using held-out measurements, while retaining original failure and sampling statistics. Do not secretly drop hard groups and report the resulting training reward as coverage.

For environment trajectories, all group members need the same task revision and comparable initial conditions. The trainer must mask tool and user tokens, track policy version over the entire episode, define terminal versus truncated returns, and monitor rollout staleness. Long episodes collected from old weights are an off-policy problem, even if the environment is fast.

Method	Required evidence or data	Main failure to watch	What a local export supplies
SFT	Reviewed demonstrations and compatible masks	Imitation of privileged or wrong behavior	Candidate conversations and lineage
DPO	Comparable chosen and rejected trajectories	Confounded or incorrect preferences	Candidate pairs and reasons
RLHF	Human judgments, reward validation, optimization	Reward-model exploitation	Feedback-compatible evidence, not a trained reward model
PPO	Fresh trajectories, behavior probabilities, advantages	Staleness, value error, wrong masks	Environment records, not training tensors
GRPO	Comparable groups, rewards, policy sampling metadata	Zero-variance groups and length bias	Groupable tasks and outcomes

Use an established trainer for optimization. verl, OpenRLHF, and Miles support distributed post-training, while RAGEN studies multi-turn agent learning and its collapse modes. Integration work remains substantial: align model templates, rollout callbacks, reward semantics, and checkpoint lifecycle. A JSONL file named "GRPO" does not complete that integration. verl, OpenRLHF, Miles, RAGEN.

Hosted APIs expose their own training and evaluation contracts. Validate a specific provider's supported dataset format and environment interaction before promising compatibility. This repository does not invoke hosted model training. OpenAI evals, OpenAI reinforcement fine-tuning.

Evaluation that supports a claim

Choose the claim before running the experiment. "The reference policy solves generated refund tasks" is an implementation check. "A trained policy solves more held-out refund tasks at equal cost" is a model experiment. "The policy reduces real support errors" requires evidence outside the simulator.

Hold fixed the model revision, policy prompt, tool schema, task revision, step budget, retry budget, sampling parameters, and execution provider. Record any exception. Compare policies on identical task identities and seeds. Randomize execution order so capacity shifts or provider incidents do not systematically favor one policy.

Report task-level outcomes and failure categories alongside aggregates. Separate policy failures, invalid tasks, environment failures, provider failures, and cancellation. A primary reliability metric can count all failures when that matches operational use. Also provide a policy-only analysis with predefined exclusions and their counts. Silently dropping failures inflates the result.

For N independent binary outcomes, estimated success is `p_hat = successes/N`. A Wilson interval behaves better than the simple normal interval for small samples and proportions near zero or one. Correlated repeats on the same task are not N independent tasks. For policy comparisons, compute per-task paired differences and bootstrap over tasks or task families, preserving repeats inside each sampled cluster. Report the confidence level and resampling protocol. The broader need to report uncertainty in few-run RL evaluation is examined in Agarwal et al..

As an illustration, 18 successes in 20 independent tasks gives 90% observed success, but a roughly 70% to 97% Wilson 95% interval. This is not evidence of a precise 90% capability. The numerical example is a statistical illustration, not a benchmark result for this build.

Pass@k measures the chance at least one of k attempts succeeds. Under independent attempts with equal success probability p, it is `1-(1-p)^k`. The common finite-sample estimator uses `1 - choose(n-c,k)/choose(n,k)` with c successful samples among n and k <= n. Pass^k, used for repeated reliability, asks that every attempt succeed and is conceptually different. Include attempt cost when comparing retries.

Prevent evaluation leakage through a locked holdout, audited access, task-family splits, and a fixed review process. A development set is where you repair prompts and graders. Repeatedly inspecting the test set and selecting the best checkpoint turns it into development data. Retain a final test set for one planned comparison or disclose the selection procedure.

Task validity has its own evaluation. Domain experts should label whether tasks are feasible, clear, representative, and correctly graded. Test a sample of claimed successes manually. Investigate unexpected failures before assuming they reveal model weakness. A policy can fail because the world omitted a necessary tool or because the grader contradicts the instruction.

Throughput, isolation, and cost

The relevant throughput is valid episodes per second or useful training tokens per unit cost, not environment steps alone. A rough capacity model is:

$$q_{\text{episodes}} \lesssim \min\left(\frac{C_{\text{env}}}{\mathbb{E}[T_{\text{episode}}]}, \frac{Q_{\text{model}}}{\mathbb{E}[n_{\text{tokens}}]}, \frac{Q_{\text{storage}}}{\mathbb{E}[n_{\text{bytes}}]}\right).$$

Here C_{env} is concurrent environment capacity, Q_{model} is effective model token throughput, and Q_{storage} is sustained trace bandwidth. Queue admission, rate limits, long-tail episodes, and failure retries reduce achieved throughput. Report startup time, model wait, tool time, grading time, persistence time, and cleanup separately.

For a local deterministic policy, model wait is nearly absent. Adding an external language model changes the bottleneck. Increasing simulator concurrency beyond inference capacity raises queue time and live-state memory without necessarily completing more episodes.

Little's law relates average in-flight work L, arrival rate lambda, and average time W through `L = lambda * W` in a stable system. It helps size a bounded queue, but it cannot justify unlimited admission. Control pending jobs, active episodes, maximum steps, observation size, and trace size independently.

Warm pools reduce cold startup latency but retain paid idle resources and create reset contamination risks. Pool only clean snapshots. Verify no prior episode files, secrets, processes, or cached tool credentials remain. An isolated database copy is cheap for a small workflow world; a full desktop snapshot has different storage and startup costs.

Containers share a kernel and require careful permissions. gVisor and microVMs add stronger execution boundaries with compatibility and cost tradeoffs. Isolation claims must name the runtime and configuration. A sandbox provider's default network access, mounted files, and secrets policy still matter.

GPU inference benefits from continuous batching, prefix caching, and enough concurrent requests to avoid idle compute. Long contexts consume KV-cache memory and can reduce concurrency. Place capture near inference, retain exact sampling metadata, and avoid cross-region request paths where latency accumulates over many turns. These are proposed deployment considerations, not features implemented by the local build.

Do not quote a universal cost per rollout. Compute:

$$\begin{aligned} C_{\text{run}} = {} & C_{\text{model input}} + C_{\text{model output}} + C_{\text{sandbox}} \\ & + C_{\text{GPU}} + C_{\text{storage}} + C_{\text{egress}} + C_{\text{retries}}. \end{aligned}$$

Measure cost per accepted trace and cost per held-out success, since a fast stream of invalid or duplicate trajectories is not useful throughput. The first external experiment should have explicit spending, token, concurrency, and duration caps, plus cancellation and cleanup proof.

Dataset quality and rights

A curated trace is a reviewed training candidate, not every recorded run. Acceptance can require correct final state, no unauthorized transitions, replay consistency, complete provenance, a permitted usage license, redaction, and a task split that avoids the holdout. Record rejection reasons so curation does not hide systematic weaknesses.

Dataset diversity should vary business rules, entity relationships, error modes, and action sequences. Renaming customers while retaining one exact solution mostly creates near-duplicates. Include ambiguous requests, permission denials, unavailable dependencies, idempotent retry cases, and situations where declining is the correct outcome.

Rights are a record-level property. Separate repository code licenses, application licenses, dataset licenses, model licenses, and customer permissions. Open source is not public domain. A public bench-

mark may permit evaluation but restrict training reuse. A contributor's permission to share a trace does not necessarily cover the personal data or copyrighted documents inside it.

Before sharing, retain a source license or consent basis, permitted uses, attribution, retention policy, redaction revision, and deletion procedure. These are data-engineering requirements, not proof of legal compliance. Obtain appropriate review for the actual jurisdiction and data agreement instead of assuming a generic notice grants rights.

A production deployment also needs authentication, authorization, per-tenant storage and encryption policies, audit logs, retention, deletion, recovery, resource quotas, and incident response. The local console is an operator tool. It should not be exposed as a multi-tenant service without that work.

A curriculum for learning and extending the system

Start with a single workflow and manually execute a successful and a failing attempt. Inspect the trusted state after each action. Write an exploit that fools a deliberately weak grade, then repair the grader and show that the same exploit fails. This establishes why outcome state and grader permissions matter.

Next, generate variations of the task and run the same policy over them. Distinguish seed variation from new rule variation. Compare reference, incomplete, and adversarial policies on the same tasks. Inspect where a shaped reward and terminal success disagree.

Then build an export manifest. Accept a reviewed successful demonstration, reject an incomplete one, and pair comparable candidates. Reproduce the accepted episode from its recorded initial state and actions. Check that the export omits private evaluator state.

Only after the held-out evaluation is stable, train a small open model with an established SFT trainer. Compare it against the base model at equal budgets. Follow with DPO if preferences add reliable information. Try grouped RL when the environment provides validated rewards and the inference stack supplies exact sampling metadata. Re-run the locked evaluation and inspect regressions, not merely mean reward.

The next implementation milestones are grounded in evidence: real application parity, broader adversarial grader review, stable held-out model evaluation, trainer integration, and a bounded cloud-provider experiment. The local model adapter is implemented; integrating it is not proof of policy improvement. The roadmap gives the required proof for each. Until those gates pass, the delivered system remains a useful local environment laboratory with explicit limits.

Practical study sequence

The companion learning labs provide six reproducible exercises with worked answers. They start by contrasting a successful tool call, a stopped episode, partial reward, and a passing grade. A refund overpayment attempt shows transition rejection, while trusted-fixture corruption independently proves the zero-reward safety gate.

The next exercises collect same-seed reference and incomplete-policy attempts, replay actions, inspect training-only split filtering, and explain why seed variation is not task-family generalization. Preference and grouped exports expose different policies, transformed trajectory completions, duplicate candidates, zero-variance reward groups, and the missing data that prevents a text trace from becoming an on-policy RL batch.

The final exercise profiles concurrency 1, 4, and 8, then contrasts simulator timing with local model inference. It examines the public Docker task's separate evaluator, artifact-only handoff, evaluation-only canary, and protocol adaptations. The objective is to learn what each measurement establishes and what additional experiment is needed for a broader claim.

Delivered capability and measured evidence

The local package delivers three synthetic workflow worlds, strict action schemas, reset and observation, isolated episode-owned Python state, trusted final grading, bounded jobs, cancellation and restart records, append-only episode storage, replay, provider adapters, candidate exports, and a connected React operator console. The process admits at most eight active episodes and 32 active runs. A nonblocking scheduler lease prevents two manager processes from recovering or scheduling the same database concurrently. The API binds to loopback and paid provider calls require both process and run opt-in.

Training formats exclude validation, test, and evaluation-only episodes and require replay consistency. SFT records controller policy-input messages per tool decision. Built-in adapters use that shared message construction, while a custom provider can transform its input internally; the controller record is not proof of a custom model's actual request. DPO retains matched provenance and labels its transformed trajectory completion; a trainer adapter is still required. Grouped candidates are explicitly nontrainable. The local provider records the installed model digest when available, but the system does not capture the complete sampled token sequence and old-policy probabilities needed for an on-policy update.

Final review found that checking transitions and the raw grader alone did not reject forged top-level reward and success labels in SDK-imported episode records. The validator now recomputes authoritative episode reward and pass, checks status/termination coherence, and verifies initial/final state, stable lineage, split, scenario, tool-context sequence, counts, and configuration-hash consistency. Regression cases reject mislabeled records before they can become accepted training candidates. This validates semantic coherence, not collection authenticity: timing, token usage, external failure causes, and asserted model identity are still reported evidence. Module and class fingerprints do not recursively attest imported external helpers; reproducibility still requires dependency and source manifests.

Split assignment uses stable environment-family/seed lineage, separate from the revision-sensitive scenario identity. This prevents a grader repair from reassigning a related seed between training and holdout. It does not hold out entire rule families. The final runner assigns zero episode reward to budget truncation, cancellation, timeout, and failed execution even when the retained diagnostic grade contains partial credit.

Synthetic workflow measurements

The raw timing matrix records 54 batches and 1,620 episode executions: three worlds, two scripted policies, three concurrency settings, three repeated trials, and 30 matched seeds per batch. On macOS arm64 with Python 3.13.5, the median end-to-end reference-policy rates were:

World	Concurrency 1, episodes/s	Concurrency 4, episodes/s	Concurrency 8, episodes/s
CRM merge	166.8	186.5	214.5
Refund operations	300.2	251.4	260.0
Incident control	176.0	172.9	178.1

All scripted reference executions passed and all incomplete-policy executions failed the task grade. Each baseline execution included a denied premature operation. There were no recorded provider or internal errors in this matrix. These rates include scheduling, hashing, grading, and SQLite persistence, not language-model inference, container provisioning, or real application execution. The incomplete policy is faster in part because it performs fewer actions; failed attempts per second are not useful-task throughput.

Each row repeats the same 30 deterministic scenarios three times. Outcome intervals use the unique first-trial scenarios rather than treating 90 repeated executions as independent evidence. The model-based Wilson 95% interval for 30/30 is approximately 88.6% to 100%; for 0/30 it is 0% to 11.4%. Even those intervals do not turn selected synthetic seeds into a representative real-world task distribution. More concurrency does not uniformly improve these CPU-heavy rates and increases per-episode latency. Cooperative scheduling does not create additional CPU execution capacity.

A same-source cache comparison bypassed only tool-schema memoization in the control and enabled it in the final matrix, preserving matched cases and all outcome counts. Across its 18 cells, observed rates improved by about 1.71 to 3.27 times. This is a local implementation measurement, not a competitor or model-inference speedup. Sequential runs had uncontrolled background load; exact fingerprints and raw timing trials are retained. The earlier pre-cache historical receipt is kept separately rather than silently replaced.

The I/O-overlap control adds a simulated 25-millisecond wait to each of six real CRM decisions. With 24 matched episodes per trial and three timing trials, median rates were 6.02, 23.75, and 46.49 episodes/s at concurrency 1, 4, and 8, with observed in-flight decision peaks 1, 4, and 8. Every attempt passed. This demonstrates overlapping awaited work in the scheduler. It is explicitly simulated latency, not measured language-model or cloud performance.

Actual local model workflow attempts

A separate paired local evaluation ran the already-installed Ollama `qwen3:8b` once per world at seed 42, alongside both scripted policies on the same scenario. It used temperature zero, thinking disabled, 512 generated-token limit per decision, 20 decisions per episode, concurrency one, and a 120-second episode timeout. Every record was evaluation-only. Collection used committed implementation revision `7fdf4c6ae511dc81cc4074ea54122c0671ba2469`, with `implementation_dirty: false`. The overall checkout was dirty because documentation and other evidence were still being assembled; the receipt does not claim a wholly clean checkout.

World	Local task grade	Termination	Invalid actions	Batch wall seconds
CRM merge	Fail	20-decision limit	0	33.24
Refund operations	Pass	Finished	1	15.13
Incident control	Fail	20-decision limit	17	34.38

The reference passed all three and the incomplete baseline failed all three. The local refund attempt first used an idempotency key belonging to another payment, received a denial, then recovered and completed the task. Its passing task grade does not qualify it for the default clean-demonstration export, which also excludes invalid actions and, independently, evaluation-only episodes. The incident

attempt repeatedly requested approval before both preflight checks passed. CRM exhausted its decision budget without completion. Neither unsuccessful attempt was a provider exception.

This nine-attempt integration sample demonstrates real model interaction, recovery from a tool denial, and distinct observed workflow failures. It is not a population estimate, policy-improvement result, or comparison with a hosted model. The model digest, actions, token counts, source fingerprints, and safe errors are preserved; token counts are not exact sampled token sequences or log probabilities. All nine full evaluation records replay against the delivered source. They are operator evidence, explicitly not training exports. Replay binds the environment implementation used at collection and intentionally rejects later source changes rather than silently regrading old records with new rules.

The earlier historical attempt remains separate. Its intermediate source was uncommitted, so the recorded fingerprints do not guarantee source recovery. The final collection repeated the same seeds without prompt tuning; these repeats do not add independent tasks. A separate UI model request was briefly submitted and cancelled around 05:03:10–05:03:15 UTC, so final inference timings may include server contention. No model-inference speedup is inferred from the historical timing difference.

One public terminal task

The separate runner executes `log-summary-date-ranges` from Terminal-Bench 2, pinned to revision `2fd12b88aafdd04a52c298e3940bcb189f9766d6`. This is a real shell and filesystem task using the original deterministic log generator and unmodified official Python tests. Its Apache 2.0 license, author attribution, source hashes, and evaluation-only canary are retained. It is not one of the synthetic workflow worlds and never enters their training exports.

The final native-tool receipt records controller revision `4b753e319fd013790524ae440d1abb9ecae9f92a` with `source_dirty: false`:

Attempt	Final official grade	Commands executed	Model calls	Wall seconds
Official reference algorithm	Pass, two tests	1	0	3.42
Deliberately wrong CSV	Fail, one of two tests	1	0	0.67
Native-tool local Qwen3 8B	Fail, empty CSV	8	8	18.27

The model was the already-installed Ollama `qwen3:8b`, Q4_K_M. Its exact model digest is:

`500a1f067a9f782620b40bee6f7b0c89e17ae61f686b92c24933e4ca4b2b8b41`

No model download or paid call occurred. The final native-tool protocol received eight valid tool calls and executed eight shell commands. The attempt exhausted its eight-call budget with no execution exception but left an empty CSV, which failed the official tests. This is an observed bounded task failure, not a malformed-action failure.

Earlier JSON-v1 protocol evidence remains separate. Its corrected-harness attempt made eight local calls over 141.36 seconds; all responses ended with `done_reason: length` and an unterminated JSON

command string. No action reached the shell. That is an action-protocol and output-budget failure. The earlier one-request harness receipt admits its raw response was not captured. Neither missing provenance nor a successful action is reconstructed. These are repeated attempts on the same task under different controller protocols, not independent task samples or evidence of changed model weights.

Agents run as a nonroot user with no container network, read-only root filesystem, dropped capabilities, no privilege escalation, CPU/memory/process limits, and a single writable CSV output mount. File size is limited to 256 KiB, core dumps are disabled, and Docker container logging is disabled. A real 512-KiB write probe failed at the 256-KiB kernel limit; the final receipt records agent and verifier limits and no remaining episode containers. Agents receive neither repository nor tests. The controller removes the agent, checks the bounded regular output artifact, then grades in a fresh container with tests mounted read-only. This is a concrete artifact handoff and evaluator boundary, not a proof against host-kernel compromise.

The official solution's package-installation bootstrap is omitted, and pytest 8.4.1 is installed during the verifier image build rather than downloaded by the official test shell script during execution. Image builds require network access; episodes and grading do not. The resolved Python base digest and agent/verifier image IDs are recorded. These disclosed adaptations are not the Terminal-Bench leaderboard protocol, and a single task with one bounded model attempt cannot establish general competence. Exact execution protocol.

What remains unproved

The final foundation verification ran 82 automated backend and public-runner tests successfully, including envelope-forgery regressions. The in-progress ATIF work is not part of this foundation claim. One upstream Starlette test-client deprecation warning remains. The browser receipt records 35 connected UI checks, no browser runtime errors, desktop and 390-pixel mobile evidence, real matched preference/grouped downloads, and cancellation through the UI. These checks support the delivered paths, not exhaustive accessibility or adversarial security coverage. Browser evidence.

No measured result establishes production application parity, human-calibrated task validity, task-family generalization, actual SFT/DPO/GRPO learning, a training improvement, hosted-provider performance, cloud isolation, multi-tenant security, or superiority to Fleet, Proximal, Harbor, or another platform. The roadmap names the required evidence rather than presenting those properties as delivered.